

Observability Stack Lab

Práctica de observabilidad sobre un sitio WordPress real, con **Prometheus + Grafana + Loki + Alloy + k6** orquestados con Docker Compose.

Stack verificado el **27 de abril de 2026** con Docker Desktop 29.x y Docker Engine 27.x.

Objetivo

Hasta ahora habéis monitoreado un servidor con `htop`, `top` y `ps`. Funcionan, pero tienen tres limitaciones serias:

1. **Solo presente.** En cuanto cierras `htop` la información se pierde.
2. **Solo una máquina.** No puedes ver 50 servidores a la vez.
3. **No hay alertas.** Si la CPU pasa al 95% mientras duermes, no te enteras.

En esta práctica vais a montar el mismo tipo de stack de observabilidad que se usa en empresas reales (Netflix, GitLab, Cloudflare): **métricas históricas, logs centralizados, dashboards y alertas automáticas**.

Al terminar, sabréis:

- Qué es **Docker** y por qué levantar 9 contenedores con un solo comando.
- Qué diferencia hay entre **métricas** (Prometheus) y **logs** (Loki).
- Cómo escribir consultas en **PromQL** y **LogQL**.
- Cómo construir un **dashboard** en Grafana y configurar **alertas**.
- Cómo simular tráfico con **k6** (sucesor moderno de Apache Benchmark).

Antes de empezar: tres conceptos

1. Métrica vs log

- **Métrica:** un número en el tiempo. Ejemplo: «CPU al 73% a las 14:35:02». Las almacena **Prometheus**.
- **Log:** una línea de texto con un evento. Ejemplo: `192.168.1.5 - - [GET /wp-admin] 200 4663`. Las almacena **Loki**.

Las dos cosas son necesarias. Una métrica te dice *que algo pasa*; un log te dice *qué pasa exactamente*.

2. Exporter

Prometheus solo entiende un formato muy concreto. Para que aprenda a leer métricas de Apache, MySQL o el sistema operativo, se usa un **exporter**: un pequeño servicio que traduce las métricas a ese formato. Vais a usar tres:

Exporter	Qué traduce
<code>node-exporter</code>	CPU, RAM, disco, red del host
<code>cadvisor</code>	CPU, RAM, red por contenedor
<code>apache-exporter</code>	Workers, peticiones, tráfico de Apache

3. Docker en una página

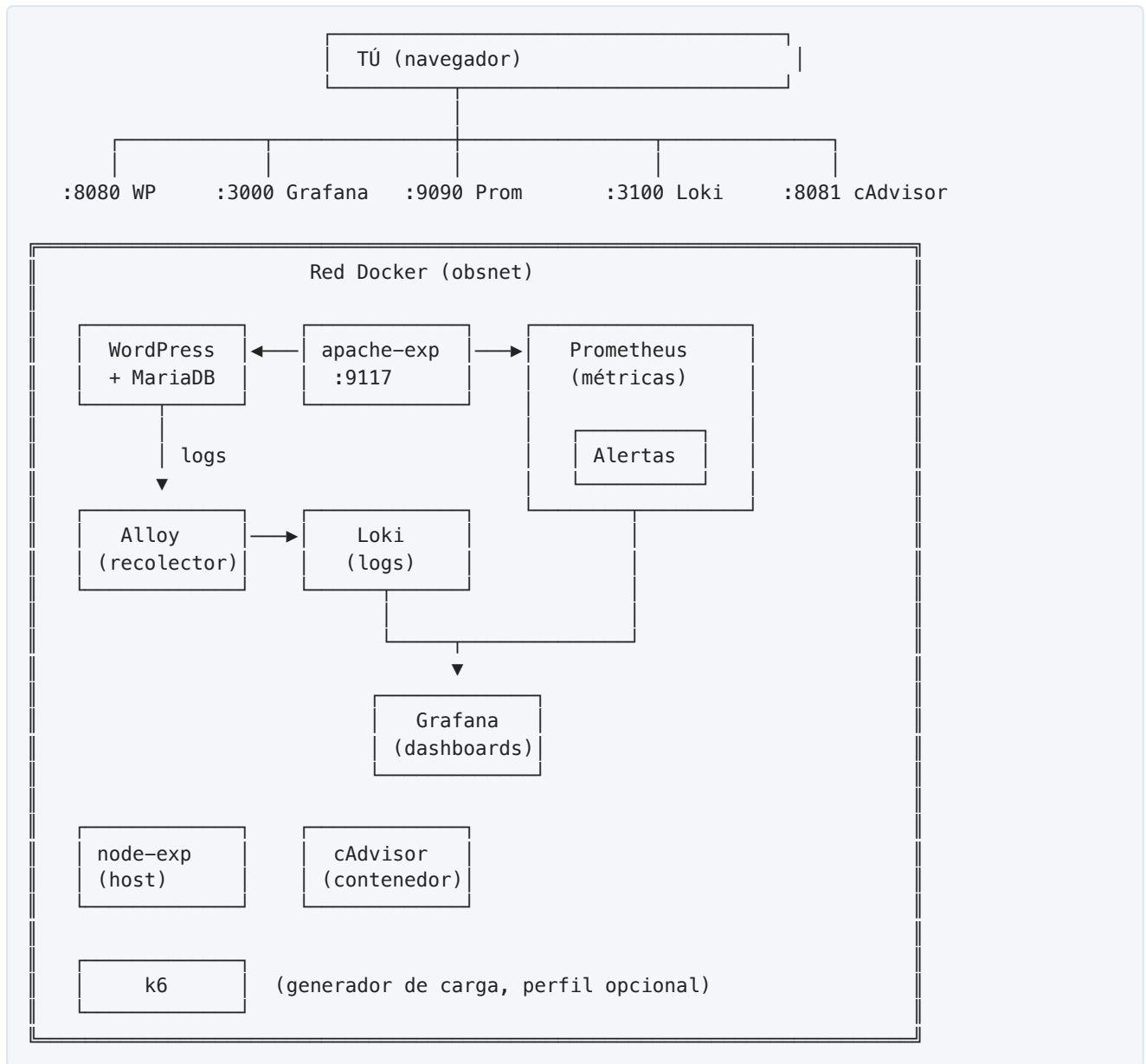
Docker es una forma de empaquetar una aplicación junto con todo lo que necesita (librerías, configuración, intérprete) en una "caja" llamada *imagen*. Cuando ejecutas la imagen obtienes un *contenedor*: un proceso aislado que se comporta como si tuviera su propio sistema operativo.

Comparación rápida:

Concepto Linux que ya conoces	Equivalente Docker
<code>apt install nginx</code>	<code>docker pull nginx</code>
<code>systemctl start nginx</code>	<code>docker run nginx</code>
<code>systemctl stop nginx</code>	<code>docker stop <id></code>
<code>journalctl -u nginx</code>	<code>docker logs <id></code>
Una VM	Un contenedor (más ligero)

Docker Compose os permite levantar **varios contenedores a la vez** con un único archivo (`docker-compose.yml`). En esta práctica vais a levantar 9 contenedores con un solo comando.

No vamos a profundizar en Docker. Si tenéis dudas, preguntad. La idea es que os centréis en la observabilidad, no en orquestar contenedores.



✓ Requisitos

- **Docker Desktop** ≥ 27.x (Mac Intel / Windows) o **Docker Engine** ≥ 27.x (Linux).
- macOS: <https://docs.docker.com/desktop/install/mac-install/>
- Windows: <https://docs.docker.com/desktop/install/windows-install/>
- Linux: <https://docs.docker.com/engine/install/>
- **8 GB de RAM** disponibles para Docker (Settings → Resources).
- **5 GB libres** en disco.
- Puertos libres: **8080** , **3000** , **9090** , **3100** , **8081** , **9100** , **9117** , **12345** .

 El stack es **multi-arch**: funciona tanto en Macs Intel como Apple Silicon (M1/M2/M3) y en Linux x86_64 / ARM64.

Comprobad que Docker funciona:

```
docker --version
docker compose version
```

Debéis ver algo como `Docker version 27.x` y `Docker Compose version v2.x`.

Estructura del proyecto

Tras clonar el repositorio veréis lo siguiente:

```
observability-stack-lab/
├── README.md                ← Este archivo
├── docker-compose.yml       ← Orquesta los 10 servicios
├── prometheus/
│   ├── prometheus.yml      ← Qué scrapea y cada cuánto
│   └── alert.rules          ← Reglas de alerta
├── alloy/
│   └── config.alloy         ← Recolector de logs Docker
├── apache/
│   └── status.conf          ← Configuración de mod_status
├── grafana/
│   └── provisioning/
│       ├── datasources/
│       │   ├── datasources.yml ← Auto-añade Prometheus y Loki
│       │   └── dashboards/
│       │       ├── dashboards.yml ← Auto-carga los JSON aquí dentro
│       └── dashboards/
└── k6/
    └── load-test.js         ← Script de generación de carga
```

Todos los archivos están listos. Solo hay que ejecutarlo.

Paso 1 · Levantar el stack

Desde la raíz del proyecto:

```
docker compose up -d
```

La primera vez Docker descargará ~2.5 GB de imágenes; tardará 3–10 minutos según vuestra conexión. Las siguientes veces tardará 15 segundos.

Cuando termine, comprobad que los 9 contenedores están arriba:

```
docker compose ps
```

Salida esperada:

NAME	STATUS	PORTS
obs-alloy	Up X seconds	0.0.0.0:12345->12345/tcp
obs-apache-exporter	Up X seconds	0.0.0.0:9117->9117/tcp
obs-cadvisor	Up X seconds (healthy)	0.0.0.0:8081->8080/tcp
obs-grafana	Up X seconds	0.0.0.0:3000->3000/tcp
obs-loki	Up X seconds	0.0.0.0:3100->3100/tcp
obs-mariadb	Up X seconds (healthy)	3306/tcp
obs-node-exporter	Up X seconds	0.0.0.0:9100->9100/tcp
obs-prometheus	Up X seconds	0.0.0.0:9090->9090/tcp
obs-wordpress	Up X seconds	0.0.0.0:8080->80/tcp

¿Algún contenedor en **Restarting** ? Mirad sus logs con `docker compose logs <nombre-servicio>` . La sección de troubleshooting al final del README cubre los problemas más típicos.

Paso 2 · Comprobar que todo funciona

Abrid estos enlaces en el navegador:

Servicio	URL	Qué deberíais ver
WordPress	http://localhost:8080	Pantalla de instalación
Grafana	http://localhost:3000	Login (usuario <code>admin</code> / pass <code>admin</code>)
Prometheus	http://localhost:9090	Interfaz de Prometheus
cAdvisor	http://localhost:8081	Métricas por contenedor
Alloy UI	http://localhost:12345	Pipeline de logs

Verificación crítica: ¿Prometheus está scrapeando todo?

1. Id a <http://localhost:9090/targets>
2. Debéis ver los **4 jobs en estado UP** (verde): - `prometheus` — Prometheus se monitorea a sí mismo - `node-exporter` — métricas del host - `cadvisor` — métricas por contenedor - `apache` — métricas de Apache

Si alguno está en **DOWN** , esperad 30 s y refrescad. Si sigue rojo → troubleshooting.

Verificación: ¿Loki está recibiendo logs?

```
curl 'http://localhost:3100/loki/api/v1/label/container/values'
```

Tenéis que ver los 9 contenedores en la lista. Si veis solo `[]` , esperad 30 s (Alloy tarda en arrancar) y volved a probar.

Paso 3 · Vuestro primer dashboard en Grafana

1. Entrad en <http://localhost:3000> (`admin` / `admin` , os pedirá cambiar la contraseña — podéis pulsar *Skip* en este lab).
2. En el menú lateral: **Dashboards** → **New** → **Import**.
3. Pegad el ID `1860` y pulsad **Load**.

4. Seleccionad la fuente de datos **Prometheus** y pulsad **Import**.

Acabáis de importar **"Node Exporter Full"**, el dashboard más popular de Grafana (131 millones de descargas). Veréis CPU, RAM, disco, red, carga, temperatura... del host.

Importad estos otros tres dashboards de la misma forma:

ID	Nombre	Qué muestra
1860	Node Exporter Full	Métricas del sistema (host)
19792	cAdvisor exporter	Métricas por contenedor
3894	Apache	Workers, req/s, tráfico
3662	Prometheus 2.0 Overview	Salud del propio Prometheus

💡 **Truco:** <https://grafana.com/grafana/dashboards/> tiene **miles** de dashboards listos. Buscad por exporter (ej. "mysql exporter") y encontraréis algo prefabricado.

🔧 Paso 4 • PromQL: vuestra primera consulta

Abrid <http://localhost:9090/graph> y pegad estas consultas (botón **Execute**):

```
# 1. Tasa de peticiones a Apache (req/s, último minuto)
rate(apache_accesses_total[1m])

# 2. ¿Cuántos workers Apache están ocupados ahora?
apache_workers{state="busy"}

# 3. Uso de CPU del host (%)
100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[2m])) * 100)

# 4. RAM libre (MB)
node_memory_MemAvailable_bytes / 1024 / 1024

# 5. Top 5 contenedores por consumo de CPU
topk(5, rate(container_cpu_usage_seconds_total{name!=""}[1m]))
```

Pulsad **Graph** (al lado de Table) para verlas como gráfica temporal.

🌟 Paso 5 • Generar carga con k6

k6 es una herramienta moderna de Grafana Labs (sucesora natural de Apache Benchmark) donde el script se escribe en JavaScript. Vais a lanzar un test que sube hasta **100 usuarios virtuales** simultáneos.

```
docker compose --profile load up k6
```

El flag **--profile load** arranca el servicio **k6** que está marcado con **profiles: ["load"]**. Sin el flag, k6 no se levanta junto al resto.

Mientras corre (~2,5 minutos), id a Grafana y abrid el dashboard **Apache**. Veréis cómo:

- `apache_workers{state="busy"}` sube de **1** a **~10**

- `rate/apache_accesses_total[1m])` salta de `0` a `100+ req/s`
- El uso de CPU del contenedor `obs-wordpress` se dispara en **cAdvisor exporter**

Cuando k6 termine, verá un resumen con métricas de p95, p99, errores, etc.

Variantes del test

Editad `k6/load-test.js` y modificad el array `stages`. Por ejemplo, para una prueba más larga y agresiva:

```
export const options = {
  stages: [
    { duration: '1m', target: 50 }, // 1 min subiendo a 50 VUs
    { duration: '3m', target: 200 }, // 3 min con 200 VUs sostenidos
    { duration: '30s', target: 0 }, // 30 s bajando
  ],
};
```

Volved a ejecutar `docker compose --profile load up k6`.

Paso 6 · Configurar y probar una alerta

Ya tenéis tres reglas de alerta cargadas en `prometheus/alert.rules`. Vedlas:

```
curl http://localhost:9090/api/v1/rules | python3 -m json.tool
```

O en la UI: <http://localhost:9090/alerts>

Las tres reglas son:

Alerta	Cuándo se dispara	Severidad
ApacheCaido	Apache no responde durante 30 s	critical
AltaCargaCPU	CPU > 80% durante 1 min	warning
MuchasPeticionesApache	Más de 50 req/s durante 30 s	info

Disparar la alerta a propósito

1. Lanzad k6: `docker compose --profile load up k6`
2. Abrid <http://localhost:9090/alerts>
3. En unos 30 s veréis `MuchasPeticionesApache` cambiar a estado `FIRING` (rojo).

Capturad esa pantalla — es uno de los entregables.

Crear vuestra propia alerta

Editad `prometheus/alert.rules` y añadid al final del grupo:

```
- alert: PocaMemoriaLibre
  expr: node_memory_MemAvailable_bytes / 1024 / 1024 < 500
  for: 1m
  labels:
    severity: warning
  annotations:
    summary: "Quedan menos de 500 MB de RAM"
```

Recargad Prometheus sin reiniciar (gracias al flag `--web.enable-lifecycle`):

```
curl -X POST http://localhost:9090/-/reload
```

Verificad en <http://localhost:9090/alerts> que aparece la nueva regla.

Paso 7 · Logs centralizados con Loki + LogQL

En Grafana, menú lateral → **Explore** → arriba seleccionad la fuente **Loki**.

Probad estas consultas en **LogQL** (es como PromQL pero para logs):

```
# Todos los logs de WordPress
{container="obs-wordpress"}

# Solo peticiones GET
{container="obs-wordpress"} |= "GET"

# Peticiones que devolvieron error 500
{container="obs-wordpress"} |~ "HTTP/1.1\" 5\\d\\d"

# Tasa de líneas de log por contenedor (último minuto)
sum by (container) (rate({container=~".+"}[1m]))

# Buscar en TODOS los contenedores menciones de "error"
{container=~".+"} |~ "(?i)error"
```

Mientras k6 está corriendo, lanzad la primera consulta y veréis los logs en **tiempo real** según WordPress los emite.

 **Por qué esto es brutal:** imaginad 50 servidores Apache. Antes teníais que `ssh` a cada uno y `tail -f /var/log/apache2/access.log`. Con Loki, una sola consulta en Grafana ve todos los logs de todos los servidores agregados.

Paso 8 · Combinar métricas y logs (la magia real)

En Grafana → **Explore** → fuente Loki, ejecutad:

```
{container="obs-wordpress"} |= "GET"
```

Pulsad el botón **Split** arriba a la derecha (icono ). En el panel nuevo, cambiad la fuente a **Prometheus** y ejecutad:

```
rate(apache_accesses_total[30s])
```

Ahora tenéis **logs y métricas sincronizados en el tiempo**. Cuando veáis un pico en la métrica, podéis mirar exactamente qué peticiones (logs) lo causaron en ese mismo instante.

Esto se llama **observabilidad**: no es solo "tener datos", es poder **investigar** sin saber de antemano qué buscar.

Entregables

Vuestra entrega debe incluir:

1. `docker-compose.yml` funcionando (con modificaciones propias si las hay).
2. `alert.rules` con la alerta extra `PocaMemoriaLibre` añadida.
3. `screenshots/` con al menos: - `targets.png` — los 4 targets de Prometheus en estado UP. - `dashboard-node.png` — Node Exporter Full mostrando datos. - `dashboard-apache.png` — Apache dashboard durante la prueba de carga (debe verse el pico en req/s). - `alert-firing.png` — `MuchasPeticonesApache` en estado FIRING. - `loki-logs.png` — Explore de Loki con logs de WordPress en directo.
4. `reporte.md` con: - **Métrica de tasa de peticiones:** `rate(apache_accesses_total[1m])` pico durante el test de carga. - **Picos de CPU / RAM** durante el test (sacar de Node Exporter Full). - **Tres consultas PromQL** que os parezcan más útiles, con explicación. - **Tres consultas LogQL** que os parezcan más útiles, con explicación. - Una **reflexión de 200 palabras:** ¿qué ventajas tiene esta stack frente a `htop` ? ¿qué le falta? (pensad en qué pasaría con 100 servidores).

Troubleshooting

Un contenedor está en `Restarting`

Mirad sus logs:

```
docker compose logs <nombre>
```

No puedo entrar a Grafana — error de credenciales

Resetead todo:

```
docker compose down -v # ¡Borra los volúmenes!
docker compose up -d
```

Las credenciales por defecto vuelven a ser `admin` / `admin`.

"address already in use" al levantar el stack

Algún puerto ya está ocupado. Buscad cuál:

```
# macOS / Linux:
lsof -nP -iTCP:8080 -sTCP:LISTEN
# Windows PowerShell:
netstat -aon | findstr :8080
```

Cambiad el mapeo de puerto en `docker-compose.yml` (ej. `"8090:80"` en vez de `"8080:80"`).

Apache exporter dice `apache_up 0`

Verificad que `mod_status` está activo:

```
docker compose exec wordpress apache2ctl -M | grep status
docker compose exec wordpress curl -s http://localhost:81/server-status?auto | head
```

La segunda debe devolver texto que empieza por `wordpress\nServerVersion: ...`.

cAdvisor sin métricas en macOS / Windows

Es **esperado**: en Docker Desktop, cAdvisor lee dentro de la VM ligera de Docker, no del Mac/Windows real. Veréis métricas por contenedor pero no del "host real". Para el lab es suficiente; en Linux nativo veríais también las métricas del host físico.

El stack consume mucha RAM

Reducid los recursos: Docker Desktop → Settings → Resources → Memory. Con 4 GB asignados a Docker funciona bien para este lab.

Quiero empezar de cero (datos limpios)

```
docker compose down -v
```

El **-v** borra los volúmenes (Prometheus, Grafana, MariaDB, WordPress).

Al terminar la sesión

Para apagar el stack pero conservar los datos:

```
docker compose down
```

Para apagar y borrar **todo** (incluidos volúmenes):

```
docker compose down -v
```

Para reanudar más tarde:

```
docker compose up -d
```

Recursos adicionales

- **Prometheus**: <https://prometheus.io/docs/introduction/overview/>
- **PromQL cheatsheet**: <https://promlabs.com/promql-cheat-sheet/>
- **Grafana**: <https://grafana.com/docs/grafana/latest/>
- **Loki + LogQL**: <https://grafana.com/docs/loki/latest/query/>
- **k6**: <https://grafana.com/docs/k6/latest/>
- **Catálogo de dashboards**: <https://grafana.com/grafana/dashboards/>
- **Catálogo de exporters de Prometheus**: <https://prometheus.io/docs/instrumenting/exporters/>

? FAQ rápida

¿Por qué **mariadb** y no **mysql**? MariaDB es un fork ligero y compatible al 100% con WordPress. La imagen oficial es más pequeña y no requiere licencia comercial para producción.

¿Por qué **Alloy** y no **Promtail**? Promtail llegó a End-of-Life en marzo de 2026. Grafana Labs lo reemplazó por Alloy, que hace lo mismo (más cosas, en realidad) y es el camino oficial hacia adelante.

¿Por qué el server-status en el puerto 81? WordPress usa `mod_rewrite` para redirigir todas las URLs no-archivo a `index.php`. Esto choca con `/server-status` (que es un handler de Apache). La solución limpia es exponer `mod_status` en un VirtualHost separado en el puerto 81 (no expuesto al host, solo accesible para el exporter dentro de la red Docker).

¿Esto se parece a producción real? Sí, mucho más de lo que parece. Las grandes empresas usan exactamente estos componentes (Prometheus, Grafana, Loki). Lo que cambia en producción es la **escala**: en vez de un Loki, tienen 50 réplicas con almacenamiento en S3. Pero los conceptos son idénticos.

💬 Si algo no funciona, no os volváis locos: revisad primero la sección de troubleshooting y luego preguntad. Es preferible resolver la duda a tiempo que pasar dos horas peleándose con un puerto ocupado.